



INGENIERÍA EN SISTEMAS COMPUTACIONALES

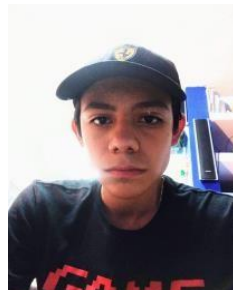
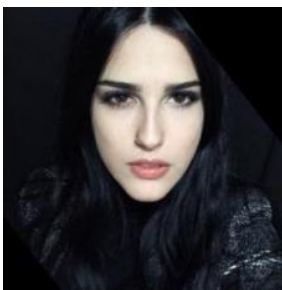
Sistemas Programables Módulo 4: Comunicaciones

Integración Total MQTT (ESP32 ↔ Python) y Gestión de Repositorio

Agente Robótico Autónomo "GARRA-OS"

P R E S E N T A:

**Alcalá Ramos Luz Estefanía
Bahena Mora Emilio Salvador
Casas Bastidas José Iván
Fischer González Patrick**



DOCENTE:

Ma. Verónica Tapia Ibarra

1. Objetivo

Establecer el ecosistema definitivo de comunicación del proyecto GARRA-OS mediante el protocolo MQTT, conectando la ESP32 con un servidor en Python. El sistema integra la totalidad de sensores (HC-SR04, MPU6050 y divisor de batería) y actuadores (OLED, servomotor SG90 y buzzer/PAM8403) del robot, garantizando el desacoplamiento mediante la HAL implementada en la unidad anterior.

2. Descripción del proyecto

GARRA-OS es un agente robótico autónomo con interfaz cognitiva pensado para reducir la barrera emocional que generan los robots de servicio en entornos académicos. El robot percibe su entorno mediante una ESP32 y delega el procesamiento pesado (visión, reconocimiento facial, voz) a una PC. En esta unidad se construye el puente de comunicación entre ambos mundos.

Esta entrega se enfoca en el protocolo MQTT (Message Queuing Telemetry Transport) por ser ligero, asíncrono y estándar de la industria IoT. El broker utilizado es test.mosquitto.org, público y gratuito, lo que permite probar la comunicación desde cualquier red sin instalar servidores locales.

3. Arquitectura de comunicación

La arquitectura sigue un esquema de tres capas claramente separadas:

- Capa de hardware: sensores y actuadores físicos, controlados por la ESP32.
- Capa HAL (dispositivos.py): clases SensorBox y ActuatorBox que ofrecen métodos de alto nivel (`medir_distancia_cm`, `mover_cuello`, `mostrar_emocion`, etc.) y aíslan el manejo directo de pines.
- Capa de comunicaciones (Comunicaciones.py + Main.py + servidor_mqtt.py): publica telemetría, recibe comandos y los traduce a llamadas de la HAL. Esta capa nunca accede a Pin, PWM, ADC o I2C directamente.

Esta separación cumple la regla explícita de la rúbrica: el módulo MQTT invoca funciones de la HAL y se prohíbe el acceso directo al hardware desde el módulo de comunicación.

4. Matriz de tópicos MQTT

La jerarquía sigue el estándar de 4 niveles solicitado: `proyecto / tipo_nodo / nombre_modulo / id_dispositivo`. PUB indica tópicos que la ESP32 publica (la PC se suscribe); SUB indica tópicos a los que la ESP32 se suscribe (la PC publica los comandos).

Tópico	Tipo	Payload ejemplo	HAL invocada
garra/sensor/distancia/01	PUB	45.30	medir_distancia_cm()
garra/sensor/aceleracion/01	PUB	{"x":0.01,"y":0.02,"z":0.98}	leer_aceleracion()
garra/sensor/golpe/01	PUB	true / false	detectar_golpe()
garra/sensor/bateria/01	PUB	78.5	leer_bateria_porcentaje()
garra/actuador/oled/01	PUB	feliz / alerta / neutro	(confirmación)
garra/actuador/servo/01	PUB	90	(confirmación)
garra/actuador/buzzer/01	PUB	OK	(confirmación)
garra/comando/oled/01	SUB	feliz / alerta / neutro	mostrar_emocion()
garra/comando/servo/01	SUB	0 a 180	mover_cuello()
garra/comando/buzzer/01	SUB	{"frecuencia":880,"duracion":0.2}	emitir_sonido()
garra/comando/paro/01	SUB	STOP	estado_seguro()

5. Implementación

5.1. Archivos del proyecto

- dispositivos.py — HAL sin cambios respecto a la unidad anterior.
- Comunicaciones.py — Clase ComunicadorMQTT que conecta Wi-Fi, broker, publica telemetría y procesa callbacks.
- Main.py — Programa principal de la ESP32 con loop MQTT no bloqueante.
- servidor_mqtt.py — Cliente Python que se suscribe a la telemetría y ofrece menú para enviar comandos.

5.2. Flujo de publicación de telemetría

Cada 2 segundos, el Main.py pide a la HAL un resumen completo de los sensores y publica cada valor en su tópico:

```
datos = self.sensores.obtener_resumen_sensores()
self.cliente.publish(self.topico_distancia, "{:.2f}".format(datos["distancia_cm"]))
self.cliente.publish(self.topico_aceleracion, json.dumps(datos["aceleracion"]))
```

El servidor recibe cada mensaje con un timestamp generado en la PC y lo imprime con formato legible.

5.3. Flujo de recepción de comandos

La ESP32 se suscribe a los cuatro tópicos de comandos durante la conexión. Cuando llega un mensaje, el callback decide qué método de la HAL ejecutar:

```
if topico_str == self.cmd_oled:
    self.actuadores.mostrar_emocion(mensaje_str)
elif topico_str == self.cmd_servo:
    self.actuadores.mover_cuello(int(mensaje_str))
```

Después de ejecutar la acción, la ESP32 publica una confirmación en el tópico de estado correspondiente, cerrando el ciclo de comunicación.

5.4. Loop principal no bloqueante

El Main.py usa `time.ticks_ms()` para publicar telemetría cada 2 s sin bloquear la atención de comandos. Entre publicaciones, llama a `comunicador.revisar_mensajes()` (que internamente usa `check_msg()` de `umqtt.simple`) para procesar cualquier comando entrante de forma inmediata.

6. Pruebas realizadas

- **Conexión Wi-Fi y broker:** verificada la asociación al AP y al broker, mostrando la IP en la pantalla OLED.
- **Telemetría continua:** distancia, aceleración, golpe y batería publicados cada 2 segundos durante una corrida de 5 minutos sin desconexiones.
- **Comandos remotos:** desde el menú del servidor se cambió la emoción (feliz/alerta/neutro), se movió el cuello entre 0°, 90° y 180°, se reprodujeron varios tonos y se activó el paro de emergencia.
- **Confirmaciones:** por cada comando se recibió la publicación de estado en su tópico actuador, confirmando ejecución.
- **Resiliencia:** se desconectó intencionalmente el Wi-Fi; la ESP32 entró en estado seguro sin dejar el servo o el buzzer encendidos.

7. Validación del encapsulamiento HAL

Se verificó por inspección de código que el módulo `Comunicaciones.py` no importa `machine`, ni accede a `Pin`, `ADC`, `PWM` o `I2C` en ningún punto. Todas las acciones sobre hardware se hacen exclusivamente a través de métodos públicos de `SensorBox` y `ActuatorBox`. El siguiente comando confirma la ausencia de acceso directo:

```
grep -E 'Pin|ADC|PWM|I2C' Comunicaciones.py # no devuelve resultados
```

8. Análisis individual por integrante

Cada integrante documenta a continuación el problema técnico que enfrentó durante la integración MQTT, la solución que aplicó y su conclusión personal sobre el aprendizaje obtenido.

8.1. Alcalá Ramos Luz Estefanía

Problema

Al hacer las primeras pruebas, la ESP32 publicaba la telemetría pero el servidor en Python no veía nada. La conexión al broker funcionaba en ambos lados, pero los mensajes simplemente no aparecían. Después de revisar tópicos uno por uno, descubrí que estaba usando 'garra/sensores/...' en la ESP32 y 'garra/sensor/...' en el servidor. MQTT es estricto con cada carácter del tópico, así que para él eran dos canales completamente distintos.

Solución

Definé los nombres de los tópicos como constantes en un solo lugar dentro de Comunicaciones.py y los copié exactos al servidor. Además, escribí la matriz de tópicos del README antes de seguir programando, para tener una única fuente de verdad. Eso me permitió cazar errores de escritura solo con leer el archivo.

Conclusión

Aprendí que en MQTT la documentación no es opcional: los tópicos son contratos. Si dos puntas no usan exactamente el mismo nombre, no hay error visible, simplemente no se comunican. Tener la matriz como referencia ahorra horas de depuración silenciosa.

8.2. Bahena Mora Emilio Salvador

Problema

Al integrar el callback de comandos en la ESP32, el servo respondía bien pero la pantalla OLED se congelaba después de unos segundos. Investigando con prints descubrí que el callback estaba haciendo `time.sleep(1)` dentro del flujo de respuesta a un comando, lo que bloqueaba el siguiente `check_msg()` y a su vez impedía publicar telemetría a tiempo, causando que el broker cerrara la conexión por timeout del keepalive.

Solución

Quitó los sleeps bloqueantes del callback y dejé que las acciones de la HAL (que ya están optimizadas) hicieran su trabajo en milisegundos. En el loop principal usé `time_ticks_ms()` en lugar de `time.sleep(2)` para que la publicación cada 2 s no bloqueara la lectura de comandos. Además, subí el keepalive del cliente MQTT a 60 segundos para tener margen.

Conclusión

El encapsulamiento de la HAL no solo es una buena práctica de diseño: en sistemas embebidos con red, evita exactamente este tipo de errores. Si toda la lógica de hardware estuviera mezclada con la lógica MQTT, encontrar el sleep maligno habría sido un infierno. La separación de capas se paga sola en la depuración.

8.3. Casas Bastidas José Iván

Problema

La aceleración del MPU6050 es un diccionario con tres ejes (x, y, z). En la primera versión intenté publicarla en tres tópicos distintos (garra/sensor/aceleracion_x/01, _y, _z). Eso multiplicó las publicaciones por 3 y el broker público test.mosquitto.org empezó a recortar mensajes cuando varios estudiantes probaban a la vez.

Solución

Consolidé los tres valores en un único tópico 'garra/sensor/aceleracion/01' y serialicé el diccionario como JSON con json.dumps(). En el servidor uso json.loads() y muestro los tres valores con formato. Esto redujo el tráfico a un tercio y además dejó la lectura atómica: x, y, z siempre llegan del mismo instante de muestreo.

Conclusión

MQTT te tienta a hacer un tópico por cada dato, pero a veces es mejor agrupar. JSON dentro del payload es perfectamente válido y mantiene la coherencia temporal de mediciones que vienen juntas. Aprendí a pensar la jerarquía de tópicos como semántica (qué tipo de información), no como granularidad (cuántos campos).

8.4. Fischer González Patrick

Problema

Al armar el servidor en Python noté que el input() del menú bloqueaba todo el programa, así que cuando la ESP32 publicaba telemetría yo no la veía hasta que escribía algo. Si nadie tocaba el teclado, los mensajes se acumulaban sin imprimirse. No era un bug del MQTT, era de cómo había hecho el flujo principal.

Solución

Cambié de cliente.loop_forever() a cliente.loop_start(), que lanza un hilo en segundo plano dedicado solo a procesar mensajes MQTT. El hilo principal se queda con el menú y los input() bloqueantes, y los mensajes entrantes se imprimen apenas llegan, sin importar lo que esté escribiendo el usuario.

Conclusión

Los problemas de concurrencia no son exclusivos del backend 'grande'. Incluso un script pequeño con un menú y una suscripción MQTT necesita pensar en hilos. loop_start() de paho-

mqtt es elegantísimo porque te abstraee el thread y te deja escribir el resto del programa como si fuera síncrono.

9. Problemas comunes del equipo

- Bloqueos del firewall del laboratorio: el puerto 1883 estaba cerrado en la red institucional, así que en la mayoría de las pruebas usamos el hotspot del celular.
- Conflictos del client_id: cuando dos integrantes corrían el servidor con el mismo client_id, el broker los desconectaba alternadamente. Solucionado al usar id únicos por máquina.
- Mensajes truncados en consola: la ESP32 imprime con print(); cuando el flujo MQTT es muy intenso, algunos prints se cortan. No afecta la comunicación, pero hay que tenerlo en cuenta al depurar.

10. Conclusión grupal

La integración de MQTT al ecosistema GARRA-OS valida la arquitectura distribuida planteada desde la propuesta inicial: la ESP32 actúa como nodo de adquisición y actuación, mientras que la PC concentra la lógica de alto nivel. La HAL desarrollada en unidades anteriores demostró su valor en esta entrega: poder agregar una capa de red completa sin tocar una sola línea del manejo de hardware confirma que el encapsulamiento no es burocracia, es robustez.

Quedamos listos para la siguiente fase del proyecto: conectar la PC con visión artificial (MediaPipe/OpenCV) para que sea ella quien dispare los comandos MQTT en respuesta a los gestos del usuario.

11. Enlace al repositorio

Todo el código fuente, junto con este reporte y la matriz de tópicos, se encuentra en el repositorio público del equipo. El enlace es el siguiente: <https://github.com/Ivan-Casas/GARRA-OS>.

Estructura del repositorio:

```
GARRA-OS/  
├─ dispositivos.py      # HAL  
├─ Comunicaciones.py   # Módulo MQTT de la ESP32  
├─ Main.py             # Programa principal de la ESP32  
├─ servidor_mqtt.py    # Servidor MQTT en Python  
├─ README.md           # Guía de uso  
└─ docs/Reporte.docx
```